

VILNIAUS UNIVERSITETAS
MATEMATIKOS IR INFORMATIKOS FAKULTETAS
PROGRAMŲ SISTEMŲ STUDIJŲ PROGRAMA

**Hibridinio genetinio paieškos algoritmo transporto
maršrutų optimizavimo uždaviniams spręsti
lygiagretinimas**

**Parallelization of Hybrid Genetic Search Algorithm for Solving
Vehicle Routing Problem**

Kursinis darbas

Atliko: 4 kurso 1 grupės studentas
Domantas Keturakis

Darbo vadovas: Doc., Dr. Algirdas Lančinskas

Vilnius – 2025

Turinys

Santrumpos	3
Įvadas	4
1. Transporto maršrutų optimizavimo uždaviniai	5
1.1. Tikslūs ir apytiksliai metodai	5
1.2. VRP variacijos	5
1.3. CVRP	5
2. HGS algoritmo veikimas	7
3. Literatūros analizė	10
3.1. Testų rinkiniai ir vertinimo metrikos	10
3.2. Lygiagretinimo kryptys	10
4. HGS-CVRP lygiagretinimas	13
5. Lygiagretintos ir nuoseklios programos palyginimas	15
5.1. Metodika	15
5.2. Eksperimentinė aplinka	15
5.3. Palyginimas	16
Rezultatai ir išvados	17
Rezultatai	17
Išvados	17
Priedai	18
1. Priedas. Eksperimento parametrai	18
2. Priedas. Vidutinės pasiektos sprendimų kainos ir spraga galutiniu laiko momentu	18
3. Priedas. Pagreitėjimo analizė	20
Šaltiniai	22

Santrumpos

Kaimynystė (*angl. neighborhood*) – sprendinių rinkinys, kurį galima gauti iš dabartinio sprendinio atlikus vieną lokalų pakeitimą (pvz., perkelti klientą, sukeisti du klientus ar apversti maršruto atkarpą).

- HGS – Hibridinis genetinis paieškos algoritmas (*angl. Hybrid Genetic Search*).
- VRP – Transporto maršrutų optimizavimo uždavinys (*angl. Vehicle Routing Problem*).
- CVRP – *angl. Capacitated Vehicle Routing Problem*. Kiekviena transporto priemonė turi maksimalią siuntų talpą.
- VRPTW – *angl. VRP with Time Windows*.
- GVRP – *angl. Generalized VRP*. Klientai grupuojami į klasterius. Tik vienas klientas iš viso klasterio turi būti aplankytas.
- CluVRP – *angl. Clustered VRP*. Klientai grupuojami į klasterius. Visi klientai klasteryje turi būti aplankyti prieš vykstant į kitą klasterį.
- SoftCluVRP – *angl. Soft Clustered VRP*. CluVRP variantas, kuriame į klasterį leidžiama aplankyti kelis kartus.
- MDVRP – *angl. Multidepot VRP*.
- PVRP – *angl. Periodic VRP*. Pridedama laiko dimensija, sprendinys sudaromas iš kelių maršrutų rinkinių, atitinkančių dienas, kuriomis bus aplankomi klientai.
- MDPVRP – *angl. Multidepot Periodic VRP*. MDVRP ir PVRP kombinacija.
- CVRPPD – *angl. CVRP Pickup and Delivery*. CVRP ir VRPPD kombinacija.
- BKS – Geriausias žinomas sprendinys (*angl. Best Known Solution*).
- GPU – Grafikos procesorius (*angl. Graphics Processing Unit*).
- Populiacija – individų rinkinys.
- Individas – uždavinio sprendinys, t. y. maršrutų rinkinys.
- Įvykdomas sprendinys – sprendinys, tenkinantis visus uždavinio apribojimus.

Įvadas

VRP – transporto maršrutų optimizavimo uždavinys [vehicle routing problem] – yra uždavinys, kurio tikslas yra surasti kuo optimaliausią maršrutų rinkinį (t. y. kuo mažiausią maršruto kainą, žr. Lygt. 2). Optimaliai parinkti maršrutai lemia, kiek klientų įmanoma aplankyti per nustatytą laiką, bei padeda sumažinti transporto kaštus. Pirmą kartą ši problema aprašyta [DR59] darbe, kur autoriai pristatė algoritmą, randantį optimalius maršrutus tarp kuro depo ir degalinių. Tai yra vienas iš modernios logistikos optimizavimo uždavinių – net maži maršrutų pagerinimai realiame pasaulyje gali reikšti reikšmingas sąnaudų ir pristatymo laiko taupymo galimybes. Keliaujančio pardavėjo uždavinyje pagrindinė užduotis yra surasti optimaliausią kelią vienam keliautojui (pardavėjui), o vrp sprendiniai susidaro iš kelių keliautojų – literatūroje šie keliautojai tiesiogiai vadinami transporto priemonėmis.

Hibridinis genetinės paieškos algoritmas (*angl. Hybrid Genetic Search – HGS*) – yra vienas iš efektyviausių genetinių metaheuristinių algoritmų [Pet+23]. Šis algoritmas ir vėlesnės pagerintos versijos išlieka etalonas daugeliui VRP variantų, „DIMACS“ konkurse [DIM22] parodęs geriausius rezultatus VRPTW uždavinyje [Koo+22], ir kurio modifikuotas variantas [Jia+22] pasirodė geriausiai CVRP uždavinyje. Šis algoritmas yra pritaikytas CVRP, VRPTW, GVRP [Lat25a], CluVRP, SoftCluVRP [Lat25b].

Šio **darbo tikslas** – išlygiagretinti hibridinio genetinio paieškos algoritmą, skirtą transporto maršrutų optimizavimo uždaviniams spręsti, siekiant sumažinti vykdymo laiką neprarandant ar net pagerinant sprendinių kokybę.

Uždaviniai:

1. Išsirinkti duomenų rinkinį, pagal kurį galima būtų testuoti ir analizuoti sprendinius.
2. Išanalizuoti, kaip veikia HGS algoritmas.
3. Atrinkti paralelizuojamas dalis, kurias galima pakeisti lygiagrečiomis.
4. Palyginti rezultatus su literatūroje aprašytais pažangiausiais algoritmais.

1. Transporto maršrutų optimizavimo uždaviniai

1.1. Tikslūs ir apytiksliai metodai

Nors egzistuoja įrankiai, pasitelkiantys tikslūs metodus (pavyzdžiui, „Google OR-Tools“ [PF25]), šis uždavinys priklauso *NP-hard* sudėtingumo klasei, todėl praktikoje dominuoja heuristikomis ir metaheuristikomis grįsti algoritmai. Tikslūs metodai (dažniausiai mišrus sveikųjų skaičių programavimas (*angl. mixed integer programming*) ar šakojimosi ir ribų (*angl. branch & bound*) metodai) suranda optimaliausius sprendinius, tačiau jų skaičiavimo laikas sparčiai auga didėjant uždavinio dydžiui. Dėl to jie tampa nepraktiški dideliems uždaviniams – pavyzdžiui, tik mažesni VRP Uždaviniai išsprendžiami tiksliais metodais per optimalų laiką. Tokie metodai dažniau taikomi mažesnėms problemoms. Tuo tarpu metaheuristiciniai algoritmai šioje uždavinių klasėje išsiskiria efektyvumu – jie per priimtina laiką randa beveik optimalius sprendinius, pasižyminčius aukšta kokybe, ir dėl to yra tinkamesni realiems logistikos uždaviniams.

Šiam uždaviniui dažniau naudojami heuristiniai ir metaheuristiciniai algoritmai, jos išlieka patrauklios realiems logistikos uždaviniams. Metaheuristika – aukštesnio lygio strategija, kuri parenka, kurias heuristikas taikyti, kad sprendiniai būtų randami efektyviau. Keli dominuojantys pavyzdžiai [Ada+24]:

- „Adaptive Large Neighborhood Search“ ir „Hybrid Adaptive Large Neighborhood Search“
- „Hybrid Genetic Search (HGS)“
- „Simulated Annealing Algorithm (SAA)“
- „Ant colony optimization (ACO)“

1.2. VRP variacijos

Egzistuoja ir VRP variacijos (CVRP, VRPTW, MDPVRP, PVRP ir kt.). Jos įveda papildomus apribojimus, pavyzdžiui: maršrutų ilgiui, transporto priemonių panaudojimo laikui ir talpai, arba prideda papildomas sąlygas:

- naudojamos transporto priemonės turi limituotą talpą (CVRP);
- visi klientai gali būti aplankyti tik specifinėmis darbo valandomis (VRPTW);
- keli depai, iš kurių galima pradėti maršrutą (MDVRP);
- maršrutai planuojami per kelias dienas, t. y. vieni klientai gali būti aplankyti vieną dieną, o kiti kitą (Periodic VRP);
- kt.

Šis darbas atsižvelgia tik į CVRP uždavinį.

1.3. CVRP

CVRP nagrinėjamas grafas $G = (V, E)$, kuriame $v_0 \in V$ žymi depą, kuris turi m transporto priemonių, o likusios viršūnės $\{v_1, \dots, v_{|V|}\}$ atitinka klientus, kuriuos reikia aplankyti. Kiekviena briauna $(i, j) \in E$ reiškia galimybę keliauti tarp vietų i ir j su kaina $c_{i,j}$ – euklidinis atstumas tarp vietų i ir j . CVRP reikia surasti sprendinį, kuriame panaudotos ne daugiau kaip K transporto priemonių, prasidedančių ir pasibaigiančių depe, taip, kad kiekvienas klientas būtų aplankytas

vieną kartą ir bendras klientų paklausos dydis bet kuriame maršrute neviršytų transporto priemonės talpos Q , o bendras transporto priemonių nuvažiuotas atstumas – kaina (žr. Lygt. 2) – būtų kiek įmanoma mažesnis.

$$c_{i,j} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \quad (1)$$

$$\text{Sprendinio kaina} = \sum_{k=1}^K \sum_{i=0}^{|V|} \sum_{j=0}^{|V|} c_{i,j} x_{i,j,k}$$

$$x_{i,j,k} = 1 \text{ Indikatorinė funkcija, kuri} \quad (2)$$

lygi 1, jei transporto priemonė k ($1 \leq k \leq K$)

keliauja nuo kliento i iki kliento j ,

lygi 0 priešingu atveju

Lyginant transporto maršrutų optimizavimo uždavinio sprendinius naudojama spraga (*angl. gap*), nusakanti atstumą procentais nuo geriausio sprendinio (*angl. Best Known Solution - BKS*) Lygt. 3.

$$\text{Spraga} = \left(\frac{Z_s - Z_{\text{BKS}}}{Z_{\text{BKS}}} \right) \cdot 100\% \quad (3)$$

Z_s = Pasirinkto algoritmo sprendinio kaina

Z_{BKS} = Geriausio sprendinio kaina

2. HGS algoritmo veikimas

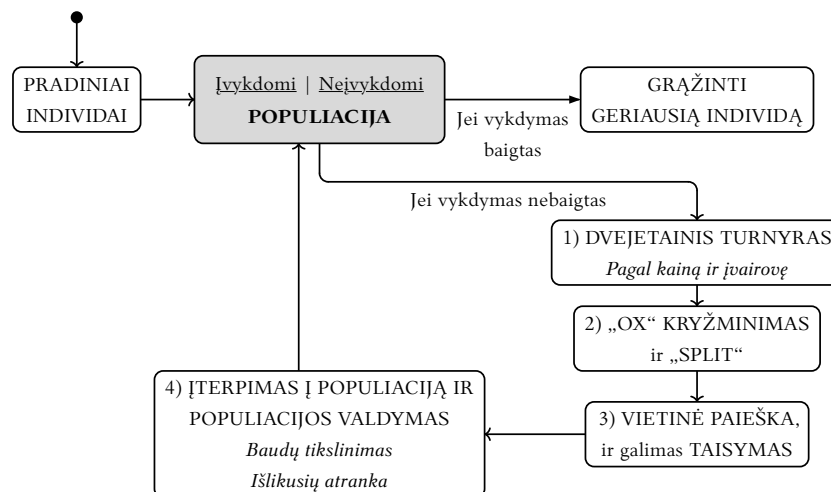
HGS algoritmas pirma skirtas MDPVRP spręsti [Vid+12]. Algoritmas patobulintas per daugelį iteracijų [Vid+14], [Vid+16], [Vid17], [Vid+21], [Vid22], ir pritaikytas CVRP. Pastarasis variantas vadinamas HGS-CVRP, o pirmasis HGS-2012.

Genetiniai algoritmai imituoja evoliucijos procesą. Populiacija yra aibė, kurią sudaro individai (t. y. užduoties sprendiniai). Šie algoritmai kombinuoja (dar vadinama kryžminimu) individus, kad sukurti naują individą palikuonį (*angl. offspring*), jį mutuoja ir prideda prie visų individų aibės – populiacijos. Kad genetinio algoritmo eiga pernelyg nesuletėtų, prasčiausi individai yra pašalinami iš populiacijos.

HGS pradinę populiaciją sukuria taikant greitas konstravimo heuristikas **kokias?**. Kryžminimo operacija sukuria vieną ilgą maršrutą, kuris vėliau efektyviai sukarponomas į kelis maršrutus pasitelkiant *Split* kaimynystės ir prideda prie populiacijos (1-ame pavyzdyje 1-as žingsnis).

Tėvų atranka palikuoniui vykdoma dvejetainiu turnyru (*angl. binary tournament*). Kryžminimui parenkami du individai kurių tinkamumas (*angl. fitness*), t. y. sprendinio kainos ir įvairovės (*broken-pairs* atstumo) suma yra didžiasios, ir išrenkami didžiausią tinkamumą turintys individai.

HGS populiacija yra sudaryta iš įvykdomų ir neįvykdomų subpopuliacijų, t. y. tų individų, kurie atitinka uždavinio apribojimus ir tų kurie jų neatitinka. Palaikant neįvykdomus (*angl. infeasible*) ir įvykdomus (*angl. feasible*) individus, išlaikoma populiacijos įvairovė (*angl. diversity*), kuri leidžia išvengti lokalaus minimumo (*angl. local minima*) beiškant geriausio individo. Neįvykdomi individai taip pat per kelias algoritmo vykdymo iteracijas gali tapti įvykdomais, todėl palaikant neįvykdomų individų subpopuliaciją, patikrinama daugiau skirtingų sprendinių.



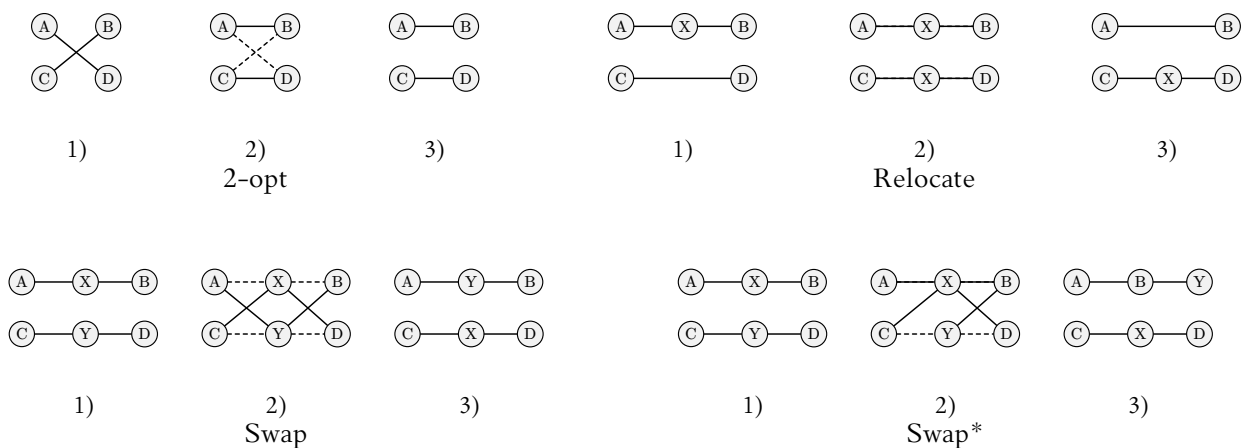
1 pav. HGS veikimas [Vid22]

HGS prie genetinio komponento prideda pagerinimo žingsnį (dar vadinama mokymo ar taisymo žingsniu) – vietinę paiešką (*angl. local search*), kuri po kryžminimo žingsnio pritaikoma naujam individui, siekiant pagerinti gauto individo kokybę (1-ame pavyzdyje 3 žingsnis).

Vietinė paieška vykdoma iteratyviai taikant kelias kaimynystes, kol nebelieka gerinančių judesių. Kaimynystės *relocate* ir *swap* leidžia keisti klientų vietas tarp maršrutų, o *2-opt* ir *2-opt** koreguoja maršruto vidinę struktūrą. *Swap** yra brangiausia, bet dažnai duodanti didžiausią

pagerėjimą kaimynystė. Esminis patobulinimas leidžiantis HGS-CVRP pasiekti vienus iš aukščiau rezultatų yra *Swap** kaimynystės pridėjimas prie prieš tai naudojamų kaimynysčių.

*Swap** kaimynystėje du klientai iš skirtingų maršrutų išimami ir kiekvienas įterpiamas į bet kurią kito maršruto poziciją; nors galimų judesių labai daug, geriausiam judesiui pakanka tikrinti įterpimą į tą pačią vietą arba į vieną iš trijų geriausių iš anksto įvertintų pozicijų, todėl kaimynystė tiriama efektyviai. Toks apribojimas sumažina vietinės paieškos sudėtingumą nuo kvadratinio iki maždaug linijinio pagal klientų skaičių, kartu išlaikant pakankamai gerą sprendinių kokybę [Vid22]. Jeigu po vietinės paieškos individas yra neįvykdomas, su 50% tikimybe taikoma taisymo procedūra (pakartotinė vietinė paieška).



2 pav. Įvairių kaimynysčių veikimas

Jeigu po vietinės paieškos individas yra neįvykdomas, algoritmas jį, su 50% tikimybe, bando taisyti (t. y. vėl taikyti vietinę paiešką) ir, priklausomai nuo rezultatų, individas yra įterpiamas į atitinkamą subpopuliaciją. Šis mechanizmas taip sutaupo CPU išteklius bei išlaiko didesnę įvairovę.

Paskutinis HGS elementas yra populiacijos valdymas, tai individų iš įvykdomų ir neįvykdomų subpopuliacijų mažiausią tinkamumą turinčių individų (t. y. didžiausios kainos ir panašių individų) pašalinimas kas numatytą iteracijų skaičių pagal baudos parametrus. Šie baudos parametrai yra kiekvieną ciklą patikslinami, kad išlaikyti balansą tarp įvykdomų ir neįvykdomų subpopuliacijų, tai vėl palaiko įvairovę ir mažina sprendinių kainą [Vid+12], [Vid22].

Svarbus niuncas yra, kad atrenkant tik geriausius individus, populiacija tampa neįvairi, t. y. visi individai yra identiški arba beveik identiški. Tai gali sukelti problemų, nes populiacija gali praleisti geriausius sprendinius, kurie nėra panašūs į esamus sprendinius populiacijoje. Šiai problemai išvengti populiacijos metu paliekami nebūtinai geriausios kainos sprendimai, bet ir tiek kurie yra labiausiai skirtingi.

- 1 Sugeneruoti pradinę populiaciją ir pagerinti ją vietine paieška
- 2 **Kol** iteracijų be pagerėjimo skaičius ir vykdymo laikas neviršija limitų: +Įterpti išmokytą
individą į atitinkamą subpopuliaciją
- 3 | Pasirinkti tėvinius individus (dvejietinis turnyras (*angl. binary tournament*))
- 4 | Atlikti kryžminimą (*angl. crossover*)
- 5 | Išmokyti naują individą (vietinė paieška)
- 6 | **Jeigu** individas neįvykdomas:
- 7 | | Su 50% tikimybe bandyti sutaisyti individą ir įtraukti į atitinkamą subpopuliaciją
- 8 | **Jeigu** pasiektas maksimalus populiacijos dydis
- 9 | | Pašalinti blogiausius ir neįvairius individus iš populiacijos
- 10 | Patikslinti baudos parametrus (*angl. penalty parameters*)
- 11 Grąžinti geriausią įvykdomą individą

3 pav. HGS algoritmo pseudokodas [Vid+12]

3. Literatūros analizė

3.1. Testų rinkiniai ir vertinimo metrikos

Algoritmų lyginimui VRP literatūroje plačiai naudojami standartizuoti testų rinkiniai su geriausiais žinomais sprendiniais (BKS). Vienas iš plačiausiai taikomų yra Uchoa 2017 CVRP rinkinys [Uch+17]. Šis uždavinių rinkinys apima platų įmanomų uždavinių skirtumus: maršruto ilgis, depo pradinė vieta, klientų pasiskirstymas ir geografinis tankumas. Todėl galima tiksliau įvertinti algoritmo charakteristikas skirtingose uždavinių tipuose.

Kadangi metaheuristikos yra stochastinės, rezultatų vertinimui paprastai naudojami vidurkiai ir geriausi pasiekti sprendiniai iš kelių paleidimų. Spraga nuo BKS išlieka svarbiausia kokybės metrika>

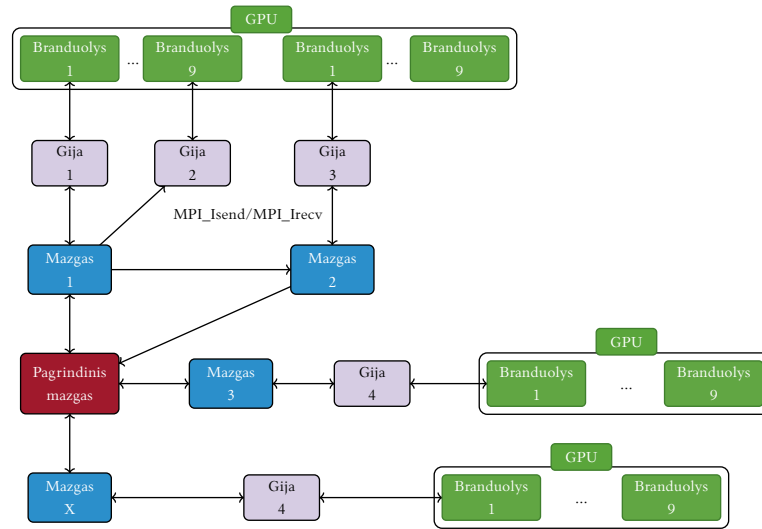
Rekomenduojama aiškiai apibrėžti laiko limitus, paleidimų skaičių ir aparatinę įrangą, nes šie veiksniai daro didelę įtaką rezultatų interpretacijai. Standartizuotos metodikos leidžia palyginti tiek algoritmo kokybę, tiek jo greitėjimo potencialą skirtingose platformose.

3.2. Lygiagretinimo kryptys

Lygiagretinimo literatūra HGS srityje orientuojasi į GPU pagrįstą skaičiavimą, kartais pasitelkiant daugiagijes CPU implementacijas. GPU sprendimai leidžia masiškai lygiagretinti kaimynystes, tačiau dažnai reikalauja supaprastinti sprendinio reprezentaciją ir mažina kaimynysčių įvairovę.

[AS20] siūlo genetinį algoritmą, kuris pilnai vykdomas GPU (CUDA): GPU branduoliai atlieka pradinę populiacijos generaciją, kainų skaičiavimą, kryžminimą, mutaciją ir *2-opt* vietinę paiešką. Sprendinių kokybei gerinti taikomos *2-opt* ir artimiausio kaimyno (*angl. nearest neighbor*) heuristikos, o autoriai pateikia CPU ir GPU versijų palyginimą bei parodo, kad *2-opt* reikšmingai mažina spragą, nors didina vykdymo laiką.

[YT21] pasitelkia GPU lygiagretinimui. Kiekvienam maršrutui skiriama atskira GPU gija, kuri vykdo vietinės paieškos žingsnį naudojant GPU. Šiuo metodu visiškai išnaudojimas priklauso nuo to, ar sukurtų maršrutų skaičius sutampa su gijų skaičiumi. Jei vietinės paieškos žingsniai modifikuoja kitų maršrutų sprendinį, gijos įrašo savo sprendinius į atskirą masyvą, kuris vėliau yra redukuojamas į vieną sprendinį, pasirenkant geriausią sprendinį. Analogiškai, vėlesniame žingsnyje kiekvienam klientui priskiriama gija. Vietinė paieška apima *swap* ir *relocate* (tarp maršrutų) bei *2-opt*, *or-opt*, *3-opt* (maršruto viduje) kaimynystes, o pradinis sprendinys konstruojamas artimiausio kaimyno metodu.



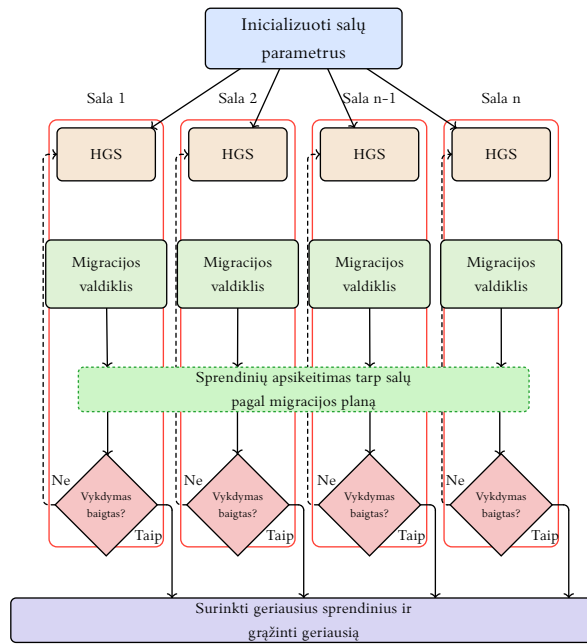
4 pav. Lygiagretintas HGS veikimui CPU ir GPU, pavyzdys iš [SND23, 11]

Vienas iš lygiagretintų HGS variantų pritaikytas pilnai išnaudoti aukšto našumo skaičiavimo (*angl. high-performance computing*) sistemas [SND23] pasitelkia tiek CPU, tiek GPU resursus (žr. 4-a pavyzdį). Šis algoritmo varinatas skirtas CVRPPD spręsti, perkelia tėvų pasirinkimo (*angl. selection*), kryžminimo (*angl. crossover*) žingsnius į atskiras CPU gijas skirtingose kompiuterių tinklo mazgose (*angl. node*). Kiekviena gija papildomai atlieka vietinę paiešką (*2-opt*, *relocate*, *swap*) pasitelkiant GPU, tačiau nepasitelkia *swap** kaimynystė. Nepaisant to, kad vieno mazgo našumas nėra pranašesnis, uždaviniai su ypač dideliais duomenų kiekiais taip gali būti greičiau apskaičiuoti negu įprastas HGS vykdomas skaičiavimus vienu procesoriumi.

[LHW25] lygiagretinimui vietinės paieškos algoritmą išreiškia tenzorių aritmetinėmis operacijomis, tai leidžia HGS vykdymą perkelti ant GPU. Taip pagreitintas vietinės paieškos operatorius. Tačiau [LHW25] pasiūlytas metodas nėra pritaikomas HGS su *swap**. „Dabartinė sprendinių reprezentacija per tensorius neleidžia lengvai įgyvendinti apkarpymo strategijų ir kaimynysčių sumažinimo technikų, kurios dažnai naudojamos vietinės paieškos grįžtais algoritmais [LHW25, 33].“¹

Priešingai nei dauguma implementacijų, „ParMDS“ naudoja grafų duomenų struktūras; panaudojami tik *2-opt* ir arčiausio kaimyno heuristikos (*angl. nearest-neighbor*). Šios heuristikos pritaikytos vykdymui GPU aplinkoje naudojant CUDA [Mun+23]. Autoriai lygina su GPU pagrįstomis genetinėmis realizacijomis ir pateikia didelį greitėjimą, tačiau jų metodas nėra HGS ir siekia greitai gauti priimtina sprendinį, o ne maksimalios kokybės sprendinį.

¹The current design of the tensor representation of solutions doesn't support easy implementation of pruning strategies and neighborhood reduction techniques that are often used in local search-based routing algorithms.



5 pav. HGS su salų modeliu [Jam+25]

PHGS (angl. *Parallel Hybrid Genetic Search*) lygiagretina HGS su salų (angl. *islands*) modeliu [Jam+25, Rez+24]. 5-ame pavyzdyje parodytas algoritmo veikimas: kiekviena gija vykdo tą patį HGS algoritmą, o migracijos žingsnis leidžia periodiškai keistis sprendiniais tarp gijų.

[WLK24] pristato „PyVRP“ paketą, kuris įgyvendina HGS algoritmą, o našumo kritinės dalis realizuoja C++ kalba. Autoriai rodo, kad tokia architektūra leidžia pasiekti mažas spragas CVRP, nors lygiagretinimas nėra pagrindinis jų tikslas. Tai pabrėžia, kad našumą galima gerinti ir per efektyvią realizaciją.

Nemaža dalis literatūros yra aprašiusi tik greitinimą ant GPU. Vis dėlto nemažos dalies pagreitinimų pritaikomumas HGS-CVRP (su *swap** operatoriumi) išlieka atvira problema.

4. HGS-CVRP lygiagretinimas

Paimta [Vid22] HGS-CVRP algoritmo implementacija², kuri naudojama kaip pagrindas lygiagretinimui.

Daugiausiai laiko užima vietinės paieškos žingsnis [Jam+25]; autoriaus matavimais vietinė paieška užima apie 86% vykdymo laiko. Todėl, siekiant sumažinti viso HGS algoritmo vykdymo laiką, šį žingsnį labiausiai verta lygiagretinti.

HGS-CVRP (su *swap** kaimynystę), pasiekia tą pačią sprendinių kokybę kaip HGS-2012 per dalį skaičiavimo laiko ir jį lenkia bet kuriame laiko taške. *Swap** paieška sudaro iki 32% vietinės paieškos CPU laiko, bet duoda apie 15% visų patobulinimų, todėl lygiagretinant svarbu šią kaimynystę išlaikyti [Vid22]. Dėl įgyvendinimo sudėtingumo ši kaimynystė dažnai praleidžiama [Jam+25, SND23].

Lygiagretinimas realizuotas naudojant „OpenMP“. Kiekvienoje iteracijoje nuosekliai veikiančioje algoritmo dalyje parenkami $2N$ tėviniai individai ir iš jų sugeneruojami N palikuonių, o vietinė paieška vykdoma lygiagrečiai – kiekviena gija apdoroja po vieną palikuonį.

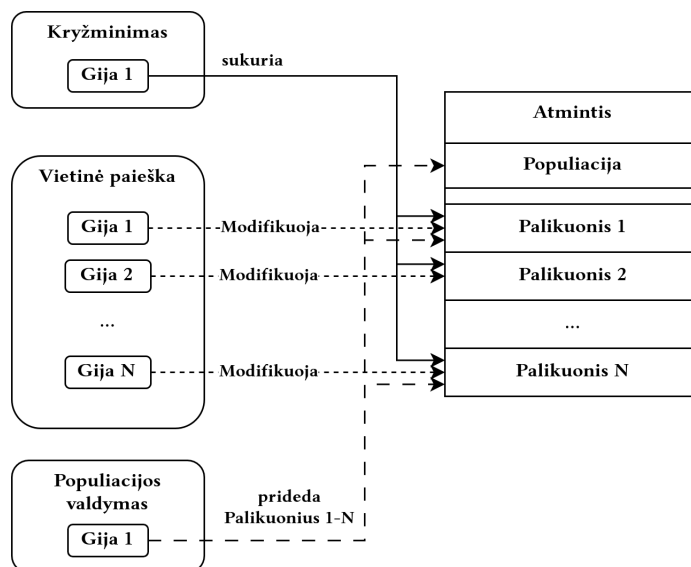
Kiekviena gija dirba su savo palikuoniu, o įrašai į bendrą populiaciją atliekami nuosekliai veikiančioje algoritmo dalyje. Toks lygiagretinimo būdas sumažina sinchronizacijos kaštus. Kiekviena gija įrašo pakeitimus tik į savo individo kopiją. Bendra populiacija atnaujinama tik nuoseklioje sekcijoje, todėl į ją patenka tik jau įvertinti individai.

Pav. 6 pavyzdys pavaizduoja, kaip kiekvienas algoritmo žingsnis modifikuoja bendrus duomenis.

Sinchronizacija vyksta pasitelkiant „OpenMP“ barjerus: po lygiagrečios vietinės paieškos ir taisymo etapų visos nuoseklios algoritmo sekcijos vykdymas blokuojamas iki tol, kol visos gijos baigia mokymo etapą. Nauji individai nuosekliai įterpiami į bendrą populiaciją.

Papildomai, šis lygiagretinimo būdas leidžia išlaikyti *swap** kaimynystę. Šis sprendimas taip pat leidžia išlaikyti HGS populiacijos valdymą vienoje vietoje ir yra paprastesnis nei GPU pagrįstas operatorių perrašymas ar salų modelio migracija.

²Nuolatinė repozitorijos nuoroda <https://github.com/vidalt/HGS-CVRP/tree/1a927955cd2861a29d978f0d359d6e647db9319c>



6 pav. Bendros atminties rašymo etapai

Greitaveiką riboja nuoseklūs žingsniai. Kryžminimo, baudų parametrų tikslinimo bei populiacijos valdymo žingsniai atliekami nuosekliai, todėl dalį laiko visos gijos, išskyrus vieną, neatlieka jokių veiksmų. Be to, prieš populiacijos valdymo žingsnį visos gijos privalo baigti vietinę paiešką, todėl lėtai veikianti gija gali užtęsti visos iteracijos vykdymo laiką.

- 1 Sugeneruoti pradinę populiaciją ir pagerinti ją vietine paieška
- 2 **Kol** iteracijų be pagerėjimo skaičius ir vykdymo laikas neviršija limitų:
- 3 | Pasirinkti $2N^3$ tėvinius individus (dvejietinis turnyras (*angl. binary tournament*))
- 4 | Atlikti kryžminimą (*angl. crossover*)
- 5 | **Kiekvienoje gijoje:**
- 6 | | Išmokyti naują individą (vietinė paieška)
- 7 | | **Jeigu** individas neįvykdomas:
- 8 | | | Su 50% tikimybe bandyti sutaisyti individą
- 9 | Įterpti išmokytus individus į atitinkamas subpopuliacijas
- 10 | **Jeigu** pasiektas maksimalus populiacijos dydis
- 11 | | Pašalinti blogiausius ir neįvairius individus iš populiacijos
- 12 | Patikslinti baudos parametrus (*angl. penalty parameters*)
- 13 Grąžinti geriausią įvykdomą individą

7 pav. Lygiagretinto HGS-CVRP pseudokodas (grįstas pagal [Vid+12])

³N – gijų skaičius

5. Lygiagretintos ir nuoseklios programos palyginimas

5.1. Metodika

HGS ir kiti iteratyvūs algoritmai sustoja pasiekus tam tikrą kriterijų. HGS atveju tai iteracijų skaičius be pagerėjimo arba veikimo laikas. Parinkus per aukštas ribas, algoritmo vykdymo laikas gali būti neprograzuojamas ir užsitęsti ilgiau negu praktiška praktikoje.

Dėl rezultatų palyginamumo pasirinkta naudoti [Vid22] aprašytus duomenų rinkinius ([Uch+17]) ir metodiką: „Mes stebime kiekvieno algoritmo pažangą iki laiko ribos $T_{\max} = n \cdot 240 / 100$ sekundžių, kur n reiškia klientų skaičių. Todėl mažiausias atvejis su 100 klientais vykdomas 4 minutes, o didžiausias atvejis su 1000 klientų vykdomas 40 minučių. Kiekvieno veikimo metu mes užregistruojame geriausią sprendimo vertę po 1%, 2%, 5%, 10%, 15%, 20%, 30%, 50%, 75% ir 100% laiko ribos, kad galėtume įvertinti algoritmų našumą skirtinguose paieškos etapuose [Vid22, 6].“⁴

Šiam eksperimentui iteracijų skaičius be pagerėjimo laikytas begaliniu, o $T_{\max} = n \cdot \frac{24}{100}$, t. y. 10 kartų mažesnis negu [Vid22], kad būtų sutilpta į MIF STSC išteklių limitus. Rezultatuose pateikiami 5 paleidimų sprendinių vykdymo laiko ir kainos vidurkiai. Lygiagretintos programos versija patalpinta „Codeberg“ repozitorijoje⁵.

Naudoti algoritmo ir vykdymo parametrai pateikti 11-ame priede. Dauguma parametrų palikti pagal numatytas reikšmes, o eksperimente fiksuoti gijų skaičius, atsitiktinės atrankos sėkla (*angl. randomization seed*) ir laiko limitas.

5.2. Eksperimentinė aplinka

Šiame darbe analizuotas Uchoa 2017 X-n uždavinių rinkinys. Palyginimui naudoti geriausių sprendinių rinkiniai⁶.

Vykdymo duomenys surinkti paleidžiant lygiagretintą ir palyginimui originalią HGS-CVRP programos versijas. Naudotodos įrangos konfigūracija: „Intel® Xeon® Gold 6252“ procesorius, 384GiB atmintis, „Qluster 13“ („Ubuntu“ pagrindas) opracinė sistema.

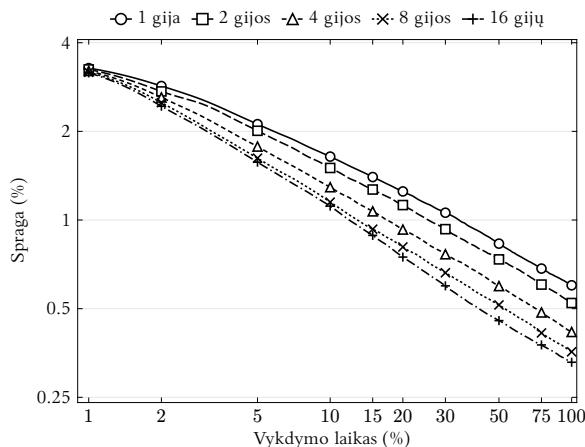
Lyginant lygiagrečią ir nuoseklią versijas naudojamas realus laikas (*angl. wall-clock*), vietoje procesoriaus laiko (*angl. CPU-time*), nes daugiagijės versijos CPU laiko matavimas parodytų kiekvieno procesoriaus vykdymo laikų sumą ir neatitiktų realaus vykdymo laiko.

⁴We monitor each algorithm's progress up to a time limit of $T_{\max} = n \cdot 240 / 100$ seconds, where n represents the number of customers. Therefore, the smallest instance with 100 clients is run for 4 minutes, whereas the largest instance containing 1000 clients is run for 40 minutes. During each run, we record the best solution value after 1%, 2%, 5%, 10%, 15%, 20%, 30%, 50%, 75%, and 100% of the time limit to measure the performance of the algorithms at different stages of the search.

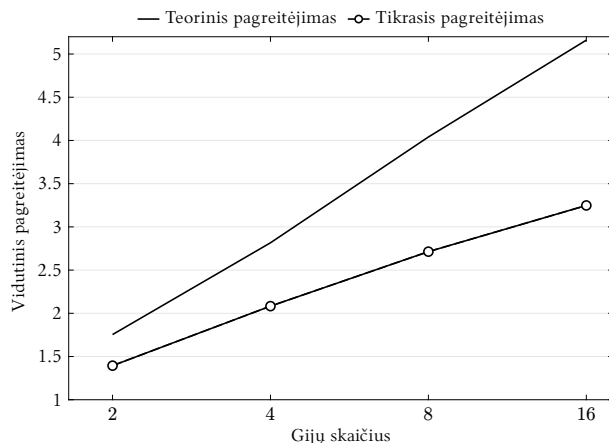
⁵<https://codeberg.org/Dom/HGS-CVRP/src/commit/411e391ffefac9a308d28e280194d65004d8332c>

⁶<https://galgos.inf.puc-rio.br/cvrplib/index.php/en/instances>, prieigos data: 2026-01-07

5.3. Palyginimas



8 pav. Vidutinė sprendinių spraga pagal gijų skaičių per vykdymo laiką (Uchoa 2017 X-n rinkinys)



9 pav. Vidutinis ir tikrasis pagreitėjimas. (Uchoa 2017 X-n rinkinys)

Lygiagreti algoritmo versija rodo mažesnę spragą ne tik vykdymo pabaigoje, bet ir ankstyvuosiuose vykdymo momentuose. Detalesni kiekvieno uždavinio rezultatai pateikti 12-oje lentelėje.

Teorinį pagreitėjimą galima matuoti pagal Amdalo dėsnį:

$$S_p^A = \frac{1}{(1-f) + \frac{f}{p}} \quad (4)$$

kur f - lygiagretinomos dalies vykdymo laikas prieš lygiagretinimą

1 lentelė. Teorinis pagreitėjimas pagal Amdalo dėsnį

p	2 gijos	4 gijos	8 gijos	16 gijos
S_p^A	1.754	2.816	4.040	5.161

Pagreitėjimui nustatyti, reikia turėti baigties stadiją. 9-ame pavyzdyje ir 2-oje lentelėje pasirinkta baigtinė stadija yra laiko momentas, kai pasiekta spraga lygi viengijos paskutinio laiko momento (t. y. kai viengijos programos veikimas baigėsi) spragai.

2 lentelė. Tikrasis pagreitėjimas

p	2 gijos	4 gijos	8 gijos	16 gijos
S_p	1.394	2.082	2.713	3.247

Greitėjimas nuo 2 iki 16 gijų auga tiesiškai. Visais atvejais pagreitėjimas nėra artimas teoriniui pagreitėjimui, bet vis tiek rodo žymų pagreitėjimą palyginus su nuoseklia programa.

Rezultatai ir išvados

Rezultatai

1. Parinktas duomenų rinkinys, pagal kurį galima testuoti ir analizuoti sprendinius.
2. Atlikta HGS algoritmo veikimo analizė ir aprašyta HGS-CVRP specifika.
3. Įgyvendintas vietinės paieškos lygiagretinimas ir aprašyta lygiagretinimo specifika.
4. Pateiktas rezultatų palyginimas.

Išvados

1. Lygiagretinta vietinė paieška HGS-CVRP algoritme, pagerina sprendinių kokybę per tą patį laiko tarpą.
2. Įmanoma lygiagretini hybridinį genetinį paieškos algortimą, kuris naudoja *swap** kaimynystę užtikrinant mažesnį vykdymo laiką.

Priedai

1. Priedas. Eksperimento parametrai

3 lentelė. Eksperimento parametrai

Parametras	Reikšmė
Algoritmo parametrai	
nbGranular (granuliarumo parametras)	20
mu (minimalus populiacijos dydis)	25
lambda (generacijos dydis)	40
nbElite (elitiniai individai)	4
nbClose (artimiausi individai)	5
nbIterPenaltyManagement (baudos parametru atnaujinimo periodas)	50
targetFeasible (tikslinė įvykdomų sprendinių dalis)	0.2
penaltyIncrease / penaltyDecrease (baudos didinimo/mažinimo koeficientai)	1.2 / 0.85
useSwapStar (swap* kaimynystės naudojimas)	1 (naudojama, kai pateiktos koordinatės)
Vykdymo parametrai	
nbIter (iteracijų limitas)	0 (iteracijų limitas netaikomas)
$T_{\max}$ (laiko limitas)	$n \cdot \frac{24}{100}$ s
seed (Atsitiktinumo sėklos)	1-5
Gijų skaičius p	1, 2, 4, 8, 16
Progreso fiksavimo žingsnis	kas 1% laiko limitu

2. Priedas. Vidutinės pasiektos sprendimų kainos ir spraga galutiniu laiko momentu

4 lentelė. Vidutinės pasiektos sprendimų kainos ir spraga galutiniu laiko momentu

Uždavinys	1 gija ⁷		2 gijos		4 gijos		8 gijos		16 gijos	
	Avg	Spraga	Avg	Spraga	Avg	Spraga	Avg	Spraga	Avg	Spraga
X-n266-k58	75856	0.5%	75802.6	0.43%	75741.4	0.35%	75707.6	0.3%	75697.8	0.29%
X-n270-k35	35316.8	0.07%	35310.8	0.06%	35317.4	0.07%	35308.2	0.05%	35315.8	0.07%
X-n275-k28	21249	0.02%	21245	0%	21245	0%	21245	0%	21245	0%
X-n280-k17	33601.4	0.29%	33595.2	0.28%	33585.8	0.25%	33600.4	0.29%	33583.4	0.24%
X-n284-k15	20314	0.44%	20286.4	0.3%	20281.8	0.28%	20265.4	0.19%	20261.8	0.18%
X-n289-k60	95755.6	0.64%	95774.8	0.66%	95519.4	0.39%	95774	0.65%	95378.2	0.24%
X-n294-k50	47252.4	0.19%	47235.6	0.16%	47216	0.12%	47220.2	0.13%	47233.6	0.15%
X-n298-k31	34270.8	0.12%	34265.2	0.1%	34263	0.09%	34269.8	0.11%	34272.8	0.12%
X-n303-k21	21811.2	0.35%	21793.2	0.26%	21786.4	0.23%	21788.8	0.24%	21788.4	0.24%
X-n308-k13	25907.8	0.19%	25935.4	0.3%	25914.8	0.22%	25889.2	0.12%	25892.6	0.13%
X-n313-k71	94289.2	0.26%	94247.6	0.22%	94165.8	0.13%	94230.2	0.2%	94177.8	0.14%
X-n317-k53	78399	0.06%	78387.6	0.04%	78401.8	0.06%	78366.2	0.01%	78368.2	0.02%

⁷Naudota originali realizacija

Uždavinys	1 gija ⁸		2 gijos		4 gijos		8 gijos		16 gijos	
	Avg	Spraga	Avg	Spraga	Avg	Spraga	Avg	Spraga	Avg	Spraga
X-n322-k28	29897.6	0.21%	29930.2	0.32%	29875.6	0.14%	29897.2	0.21%	29885.4	0.17%
X-n327-k20	27611	0.29%	27580.8	0.18%	27580.4	0.18%	27584	0.19%	27605.4	0.27%
X-n331-k15	31157.4	0.18%	31160.2	0.19%	31116	0.05%	31112.4	0.03%	31104.2	0.01%
X-n336-k84	140202.6	0.78%	139885	0.56%	139838	0.52%	139712.4	0.43%	139448.6	0.24%
X-n344-k43	42145.6	0.23%	42108.6	0.14%	42111.4	0.15%	42149.2	0.24%	42113.4	0.15%
X-n351-k40	25990	0.36%	25982	0.33%	25973.4	0.3%	25979.4	0.32%	25999.4	0.4%
X-n359-k29	51896.2	0.76%	52020.2	1%	51752.2	0.48%	51711.6	0.4%	51726.8	0.43%
X-n367-k17	22819.2	0.02%	22814	0%	22815.6	0.01%	22819.8	0.03%	22815.4	0.01%
X-n376-k94	147796.6	0.06%	147788.8	0.05%	147734	0.01%	147754.2	0.03%	147730.8	0.01%
X-n384-k52	66260	0.49%	66158.2	0.33%	66145.8	0.31%	66120.6	0.27%	66128.6	0.29%
X-n393-k38	38294.8	0.09%	38277.6	0.05%	38308.4	0.13%	38284.6	0.06%	38293.6	0.09%
X-n401-k29	66423.8	0.41%	66334.4	0.27%	66300.4	0.22%	66253.6	0.15%	66280.8	0.19%
X-n411-k19	19746.8	0.18%	19748.4	0.18%	19735.6	0.12%	19720	0.04%	19729.8	0.09%
X-n420-k130	107963.6	0.15%	107916.2	0.11%	107918.4	0.11%	107878.4	0.07%	107934.6	0.13%
X-n429-k61	65578.8	0.2%	65607.8	0.24%	65585.2	0.21%	65523.6	0.11%	65576.4	0.19%
X-n439-k37	36422.4	0.09%	36414.4	0.06%	36411.8	0.06%	36424.8	0.09%	36419	0.08%
X-n449-k29	55850.8	1.12%	55659	0.77%	55565	0.6%	55507	0.5%	55483.4	0.45%
X-n459-k26	24212.2	0.3%	24191.4	0.22%	24201.6	0.26%	24182.6	0.18%	24184.8	0.19%
X-n469-k138	223620	0.81%	223305.8	0.67%	223117.8	0.58%	222741.2	0.41%	222988.4	0.52%
X-n480-k70	89756.2	0.34%	89685.2	0.26%	89623.6	0.2%	89588.8	0.16%	89554.8	0.12%
X-n491-k59	66997.6	0.77%	66886.2	0.61%	66848.8	0.55%	66772.6	0.44%	66683.6	0.3%
X-n502-k39	69324	0.14%	69307.2	0.12%	69278.4	0.08%	69273.2	0.07%	69267.6	0.06%
X-n513-k21	24220.4	0.08%	24226.4	0.1%	24222	0.09%	24230.8	0.12%	24208	0.03%
X-n524-k153	154951.4	0.23%	154982.4	0.25%	154963.4	0.24%	155021.2	0.28%	154950.2	0.23%
X-n536-k96	95547.8	0.74%	95361.8	0.54%	95255.4	0.43%	95258	0.43%	95234.2	0.41%
X-n548-k50	87053	0.41%	86928	0.26%	86851.4	0.17%	86870.4	0.2%	86829.8	0.15%
X-n561-k42	42808	0.21%	42815	0.23%	42807.4	0.21%	42774.4	0.13%	42764.6	0.11%
X-n573-k30	51087.4	0.82%	51045.4	0.73%	50975	0.6%	50900.8	0.45%	50860.2	0.37%
X-n586-k159	191509.8	0.63%	191373.4	0.56%	191186.2	0.46%	190922	0.32%	190897	0.31%
X-n599-k92	109069.4	0.57%	109017.2	0.52%	109057.8	0.56%	108882.6	0.4%	108870.8	0.39%
X-n613-k62	59975	0.74%	59869.8	0.56%	59745.6	0.35%	59844.4	0.52%	59817.2	0.47%
X-n627-k43	62869.4	1.13%	62850.4	1.1%	62728.2	0.91%	62655.2	0.79%	62562.8	0.64%
X-n641-k35	64464.6	1.23%	64294	0.96%	64047.4	0.57%	64069.4	0.61%	63945.4	0.41%
X-n655-k131	106924.6	0.14%	106892.8	0.11%	106846.2	0.06%	106831	0.05%	106841.2	0.06%
X-n670-k130	147471.2	0.78%	147367.2	0.71%	147195.8	0.59%	147165.2	0.57%	147041.2	0.48%
X-n685-k75	68725	0.76%	68592.4	0.57%	68502.2	0.44%	68482.8	0.41%	68484.6	0.41%
X-n701-k44	83180.6	1.54%	83065.4	1.39%	82578.4	0.8%	82613.8	0.84%	82503	0.71%
X-n716-k35	43900.2	1.22%	43772.8	0.92%	43728.8	0.82%	43567.6	0.45%	43574	0.46%
X-n733-k159	136768	0.43%	136792	0.44%	136717.8	0.39%	136566.4	0.28%	136598.8	0.3%
X-n749-k98	78401.6	1.47%	78411.4	1.48%	78206.2	1.21%	78064.2	1.03%	77968.8	0.91%
X-n766-k71	115315.2	0.79%	115179.4	0.67%	115114.4	0.61%	114935.8	0.45%	114904.2	0.43%

⁸Naudota originali realizacija

Uždavinys	1 gija ⁹		2 gijos		4 gijos		8 gijos		16 gijos	
	Avg	Spraga	Avg	Spraga	Avg	Spraga	Avg	Spraga	Avg	Spraga
X-n783-k48	73721.4	1.84%	73577	1.65%	73425.4	1.44%	73074.6	0.95%	72998.6	0.85%
X-n801-k40	74229.8	1.25%	74182.6	1.19%	73988.2	0.92%	73847	0.73%	73724.2	0.56%
X-n819-k171	159540.4	0.9%	159153.2	0.65%	158976.2	0.54%	158891.2	0.49%	158885.2	0.48%
X-n837-k142	196146	1.24%	195816.2	1.07%	195501	0.91%	195255.2	0.78%	195175.4	0.74%
X-n856-k95	89094.2	0.15%	89112.2	0.17%	89093	0.14%	89044.6	0.09%	89101	0.15%
X-n876-k59	100509.8	1.22%	100395.4	1.1%	100196.4	0.9%	100084.4	0.79%	100048.8	0.76%
X-n895-k37	54604.8	1.38%	54511.8	1.21%	54272.4	0.77%	54208	0.65%	54249.4	0.72%
X-n916-k207	332510.6	1.01%	332303.2	0.95%	332145.8	0.9%	331893.8	0.82%	331591.6	0.73%
X-n936-k151	133835	0.84%	133767	0.79%	133855.8	0.86%	133585.6	0.66%	133661.4	0.71%
X-n957-k87	86039	0.67%	85958.4	0.58%	85765.2	0.35%	85721.8	0.3%	85681.8	0.25%
X-n979-k58	120554	1.33%	120308.2	1.12%	120170.2	1%	119805.4	0.7%	119821.6	0.71%
X-n1001-k43	73938.4	2.19%	73748.4	1.93%	73368.4	1.4%	73202	1.17%	73232.8	1.21%

3. Priedas. Pagreitėjimo analizė

5 lentelė. Pagreitėjimas kiekvienam uždaviniui.

Uždavinys	2 gijos	4 gijos	8 gijos	16 gijos
X-n266-k58	1.23	1.52	1.75	2.78
X-n270-k35	1.39	1	2.27	2.86
X-n275-k28	2	5.26	4.55	5
X-n280-k17	1.27	1.72	1.03	2.94
X-n284-k15	2	2.13	2.56	3.33
X-n289-k60	1	1.85	1	2.7
X-n294-k50	1.89	5.56	7.69	4
X-n298-k31	2.17	3.13	1.79	1
X-n303-k21	1.52	2	2.63	2.27
X-n308-k13	1	1	2	2.78
X-n313-k71	1.27	1.89	1.79	2.7
X-n317-k53	1.19	1	2.56	3.23
X-n322-k28	1	4	1.12	4.35
X-n327-k20	1.69	2.94	4.17	1.39
X-n331-k15	1	2.08	2.22	7.14
X-n336-k84	1.45	1.85	1.92	3.7
X-n344-k43	2.22	2.27	1	3.33
X-n351-k40	1.1	2.27	2.63	1
X-n359-k29	1	2.17	3.23	3.45
X-n367-k17	1.89	1.28	1	2.86
X-n376-k94	1.06	2.13	2	2.38
X-n384-k52	1.82	2	2.5	2.94
X-n393-k38	2.04	1	1.41	1.25
X-n401-k29	1.47	2.17	4.55	3.57

⁹Naudota originali realizacija

Uždavinys	2 gijos	4 gijos	8 gijos	16 gijos
X-n411-k19	1	1.72	5.26	5
X-n420-k130	1.61	2.08	2.94	1.72
X-n429-k61	1	1	2.63	1.14
X-n439-k37	1.54	3.13	1	1.67
X-n449-k29	1.47	2	3.45	5
X-n459-k26	1.37	1.39	2.22	4
X-n469-k138	1.61	2.33	4.76	3.03
X-n480-k70	1.27	2	2.78	3.7
X-n491-k59	1.37	1.69	2.56	4.17
X-n502-k39	1.28	2.08	3.03	3.45
X-n513-k21	1	1	1	4
X-n524-k153	1	1	1	1.16
X-n536-k96	1.82	2	2.78	3.13
X-n548-k50	1.54	2.27	2.63	4
X-n561-k42	1	1.06	2.86	3.33
X-n573-k30	1.19	2.38	3.33	5.88
X-n586-k159	1.27	1.69	2.44	2.17
X-n599-k92	1.08	1.03	1.41	2.33
X-n613-k62	1.56	3.85	3.85	3.7
X-n627-k43	1.09	1.56	2.5	3.03
X-n641-k35	1.69	2.7	2.94	4.55
X-n655-k131	1.43	2.63	3.45	2.63
X-n670-k130	1.33	2.5	2.5	3.45
X-n685-k75	1.82	3.57	4.35	4.35
X-n701-k44	1.2	2.63	3.33	4.17
X-n716-k35	1.61	2.13	4.35	5.26
X-n733-k159	1	1.08	2.44	1.79
X-n749-k98	1	1.43	1.92	2.44
X-n766-k71	1.47	2.17	3.33	4.35
X-n783-k48	1.39	1.96	3.57	3.7
X-n801-k40	1.14	2.04	2.86	4.35
X-n819-k171	1.67	2.33	2.44	2.56
X-n837-k142	1.72	2.33	3.13	4.17
X-n856-k95	1	1.03	2.27	1
X-n876-k59	1.28	2	2.56	4
X-n895-k37	1.32	2.56	3.33	3.23
X-n916-k207	1.19	1.39	1.85	3.13
X-n936-k151	1.52	1	2.13	2.5
X-n957-k87	1.22	2.33	2.78	4
X-n979-k58	1.56	1.69	4.17	4
X-n1001-k43	1.33	2.38	2.86	2.94

Šaltiniai

- [AS20] M. F. Abdelatti ir M. S. Sodhi, „An improved GPU-accelerated heuristic technique applied to the capacitated vehicle routing problem“, *Proceedings of the 2020 Genetic and Evolutionary Computation Conference*, GECCO '20. ACM, birž. 2020, p. 663–671. doi: [10.1145/3377930.3390159](https://doi.org/10.1145/3377930.3390159).
- [Ada+24] T. Adamo, M. Gendreau, G. Ghiani, ir E. Guerriero, „A review of recent advances in time-dependent vehicle routing“, *European Journal of Operational Research*, t. 319, nr. 1, p. 1–15, lapkr. 2024, doi: [10.1016/j.ejor.2024.06.016](https://doi.org/10.1016/j.ejor.2024.06.016).
- [DR59] G. B. Dantzig ir J. H. Ramser, „The Truck Dispatching Problem“, *Management Science*, t. 6, nr. 1, p. 80–91, spal. 1959, doi: [10.1287/mnsc.6.1.80](https://doi.org/10.1287/mnsc.6.1.80).
- [DIM22] DIMACS, „The 12th DIMACS Implementation Challenge: Vehicle Routing Problems (VRP)“, 2022 m.
- [Jam+25] M. Jamshidi, M. Alirezanejad, H. Motameni, ir R. Enayatifar, „A Parallel Hybrid Genetic Search for Solving the Capacitated Vehicle Routing Problem“, *International Journal of Computational Intelligence Systems*, t. 18, nr. 1, lapkr. 2025, doi: [10.1007/s44196-025-01059-0](https://doi.org/10.1007/s44196-025-01059-0).
- [Jas+24] T. Jastrzab ir kt., „Standardized validation of vehicle routing algorithms“, *Applied Intelligence*, t. 54, nr. 2, p. 1335–1364, saus. 2024, doi: [10.1007/s10489-023-05212-0](https://doi.org/10.1007/s10489-023-05212-0).
- [Jia+22] S. Jiang, Z. He, W. Lin, F. Ma, ir Z. Lü, „FHCSolver: Fast hybrid CVRP solver“, *12th DIMACS implementation challenge: Vehicle routing problems*, 2022.
- [Koo+22] W. Kool ir kt., „Hybrid genetic search for the vehicle routing problem with time windows: A high-performance implementation“, *12th dimacs implementation challenge workshop*, 2022.
- [LHW25] Z. Lei, J.-K. Hao, ir Q. Wu, „Speeding up Local Optimization in Vehicle Routing with Tensor-based GPU Acceleration“. [Interaktyvus]. Adresas: <https://arxiv.org/abs/2506.17357>
- [Lat25] V. Latorre, „A hybrid genetic search based approach for the generalized vehicle routing problem“, *Soft Computing*, t. 29, nr. 3, p. 1553–1566, vas. 2025a, doi: [10.1007/s00500-025-10507-0](https://doi.org/10.1007/s00500-025-10507-0).
- [Lat25] V. Latorre, „An application of a two-level genetic search for the soft-clustered vehicle routing problem“, *Evolutionary Intelligence*, t. 18, nr. 4, liep. 2025b, doi: [10.1007/s12065-025-01063-5](https://doi.org/10.1007/s12065-025-01063-5).
- [Mun+23] R. P. Muniasamy, S. Singh, R. Nasre, ir N. Narayanaswamy, „Effective Parallelization of the Vehicle Routing Problem“, *Proceedings of the Genetic and Evolutionary Computation Conference*, GECCO '23. ACM, liep. 2023, p. 1036–1044. doi: [10.1145/3583131.3590458](https://doi.org/10.1145/3583131.3590458).
- [PF25] L. Perron ir V. Furnon, „OR-Tools“. [Interaktyvus]. Adresas: <https://developers.google.com/optimization/>
- [Pan+23] R. Pandian M, S. Singh, R. Nasre, ir N. S. Narayanaswamy, „Effective Parallelization of the Vehicle Routing Problem“, *Proceedings of the Genetic and Evolutionary*

- Computation Conference (GECCO '23)*, New York, NY, USA: ACM, 2023. doi: [10.1145/3583131.3590458](https://doi.org/10.1145/3583131.3590458).
- [Pet+23] F. Petropoulos *ir kt.*, „Operational Research: methods and applications“, *Journal of the Operational Research Society*, t. 75, nr. 3, p. 423–617, gruodž. 2023, doi: [10.1080/01605682.2023.2253852](https://doi.org/10.1080/01605682.2023.2253852).
- [WLK24] N. A. Wouda, L. Lan, ir W. Kool, „PyVRP: A High-Performance VRP Solver Package“, *INFORMS Journal on Computing*, t. 36, nr. 4, p. 943–955, liep. 2024, doi: [10.1287/ijoc.2023.0055](https://doi.org/10.1287/ijoc.2023.0055).
- [Rez+24] B. Rezaei, F. Gadelha Guimaraes, R. Enayatifar, ir P. C. Haddow, „Exploring dynamic population Island genetic algorithm for solving the capacitated vehicle routing problem“, *Memetic Computing*, t. 16, nr. 2, p. 179–202, birž. 2024, doi: [10.1007/s12293-024-00412-8](https://doi.org/10.1007/s12293-024-00412-8).
- [SND23] T. Stadtler, S. Nita, ir J. Dünnweber, „A parallel hybrid genetic search for the capacitated vrp with pickup and delivery“, Ostbayerische Technische Hochschule Regensburg, 2023.
- [Uch+17] E. Uchoa, D. Pecin, A. Pessoa, M. Poggi, T. Vidal, ir A. Subramanian, „New benchmark instances for the Capacitated Vehicle Routing Problem“, *European Journal of Operational Research*, t. 257, nr. 3, p. 845–858, kovo 2017, doi: [10.1016/j.ejor.2016.08.012](https://doi.org/10.1016/j.ejor.2016.08.012).
- [Vid+12] T. Vidal, T. G. Crainic, M. Gendreau, N. Lahrichi, ir W. Rei, „A Hybrid Genetic Algorithm for Multidepot and Periodic Vehicle Routing Problems“, *Operations Research*, t. 60, nr. 3, p. 611–624, birž. 2012, doi: [10.1287/opre.1120.1048](https://doi.org/10.1287/opre.1120.1048).
- [Vid+14] T. Vidal, T. G. Crainic, M. Gendreau, ir C. Prins, „A unified solution framework for multi-attribute vehicle routing problems“, *European Journal of Operational Research*, t. 234, nr. 3, p. 658–673, geg. 2014, doi: [10.1016/j.ejor.2013.09.045](https://doi.org/10.1016/j.ejor.2013.09.045).
- [Vid+16] T. Vidal, N. Maculan, L. S. Ochi, ir P. H. Vaz Penna, „Large Neighborhoods with Implicit Customer Selection for Vehicle Routing Problems with Profits“, *Transportation Science*, t. 50, nr. 2, p. 720–734, geg. 2016, doi: [10.1287/trsc.2015.0584](https://doi.org/10.1287/trsc.2015.0584).
- [Vid+21] T. Vidal, R. Martinelli, T. A. Pham, ir M. H. Hà, „Arc Routing with Time-Dependent Travel Times and Paths“, *Transportation Science*, t. 55, nr. 3, p. 706–724, geg. 2021, doi: [10.1287/trsc.2020.1035](https://doi.org/10.1287/trsc.2020.1035).
- [Vid16] T. Vidal, „Technical note: Split algorithm in $O(n)$ for the capacitated vehicle routing problem“, *Computers & Operations Research*, t. 69, p. 40–47, 2016, doi: <https://doi.org/10.1016/j.cor.2015.11.012>.
- [Vid17] T. Vidal, „Node, Edge, Arc Routing and Turn Penalties: Multiple Problems—One Neighborhood Extension“, *Operations Research*, t. 65, nr. 4, p. 992–1010, rugpj. 2017, doi: [10.1287/opre.2017.1595](https://doi.org/10.1287/opre.2017.1595).
- [Vid22] T. Vidal, „Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood“, *Computers & Operations Research*, t. 140, p. 105643, bal. 2022, doi: [10.1016/j.cor.2021.105643](https://doi.org/10.1016/j.cor.2021.105643).

- [YT21] P. Yelmewad ir B. Talawar, „Parallel Version of Local Search Heuristic Algorithm to Solve Capacitated Vehicle Routing Problem“, *Cluster Computing*, t. 24, nr. 4, p. 3671–3692, liep. 2021, doi: [10.1007/s10586-021-03354-9](https://doi.org/10.1007/s10586-021-03354-9).